

360° Vision Pipeline

A ground-up technical explanation of the nuScenes data pipeline — coordinate transforms, LiDAR projection, and bird's-eye-view rendering.

CHAPTER 1

The nuScenes Dataset — What We're Working With

nuScenes is a real autonomous driving dataset collected in Boston and Singapore. A vehicle drove around with six cameras pointing in every direction (front-left, front, front-right, back-left, back, back-right) and one spinning LiDAR on the roof. Everything was recorded, timestamped, and carefully calibrated.

The dataset is split into **850 scenes**. Each scene is an independent 20-second clip of urban driving with no relation to the scenes before or after it. Each scene contains roughly 40 *key samples*, captured at 2 Hz — one snapshot every half second. A snapshot means: one LiDAR sweep + one image from each of the six cameras, all taken at approximately the same moment.

The raw sensor data is also stored between keyframes (cameras fire at ~12 Hz, LiDAR at ~20 Hz). These intermediate captures are called *sweeps*.

How the data is stored on disk

Everything lives in JSON files. nuScenes is built around a set of database-style tables, where every record has a unique **token** (a UUID string) and records reference each other by token:

File	What it contains
scene.json	One entry per driving clip. Has the scene name and the token of the first sample.
sample.json	One entry per keyframe. Has a <code>next</code> and <code>prev</code> token forming a linked list through the scene.
sample_data.json	One entry per actual file (image or .pcd.bin). References the sample it belongs to and the sensor it came from.
calibrated_sensor.json	The physical position and orientation of each sensor on the car — its translation and rotation relative to the ego vehicle.
ego_pose.json	Where the car was in the world (GPS/IMU pose) at a given timestamp.

To walk through all keyframes in a scene, you start with `scene["first_sample_token"]`, look it up in `sample.json`, process it, then follow `sample["next"]` to the next one, and so on until `next` is empty. This is exactly what `_sample_sequence()` in `data.py` does.

The LiDAR point cloud is stored as a raw binary file (.pcd.bin). Each point is five float32 values: x, y, z, intensity, and ring index. We load it with `np.fromfile(..., dtype=np.float32).reshape(-1, 5)[: , :3]` — we only keep x, y, z.

CHAPTER 2

Coordinate Frames — The Foundation of Everything

This is the single most important concept to understand before anything else makes sense.

Every sensor on the car lives in its own **coordinate frame** — its own local sense of "where is x, y, z". When a LiDAR point is at position (10, 3, 0), that's 10 metres ahead, 3 metres to the left, and 0 metres up *relative to the LiDAR sensor*. The camera has no idea what that means until you convert it into the camera's own coordinate frame.

There are four frames in play:

Frame	Origin	Used for
LiDAR sensor frame	Centre of the spinning LiDAR on the roof	Raw point cloud coordinates
Ego vehicle frame	Centre of the car	Intermediate step; the BEV lives here
World frame	Fixed GPS origin	Intermediate step when moving between sensors at different timestamps
Camera sensor frame	Centre of each camera's lens	Final step before projecting to pixels

nuScenes calibrates all of these precisely. For each sensor, it tells you: "this sensor is at translation **t** (metres) and rotation **q** (quaternion) relative to the ego vehicle." And for each timestamp, it tells you: "the ego vehicle was at translation **t** and rotation **q** in the world."

To go from one frame to another, you apply a rigid-body transform — a rotation followed by a translation. Let's understand that math first.

Quaternions and Rotation Matrices

Rotations in 3D space are stored as **quaternions** in nuScenes. A quaternion is a compact 4-number representation of any 3D rotation: (w, x, y, z) .

You can think of it this way: any rotation in 3D can be described as "rotate by angle θ around axis (ax, ay, az) ". The quaternion encodes this as:

```
w = cos(theta/2)
x = ax * sin(theta/2)
y = ay * sin(theta/2)
z = az * sin(theta/2)
```

Example: 90° rotation around the Z-axis (yawing left)

$\theta = 90^\circ$, axis = $(0, 0, 1)$

$w = \cos(45^\circ) = 0.707$

$x = 0 \cdot \sin(45^\circ) = 0$

$y = 0 \cdot \sin(45^\circ) = 0$

$z = 1 \cdot \sin(45^\circ) = 0.707$

So the quaternion is **$(0.707, 0, 0, 0.707)$**

To actually rotate a point, we convert the quaternion to a **3×3 rotation matrix**. This is what

`quaternion_to_rotation_matrix()` computes:

Given unit quaternion (w, x, y, z) , the rotation matrix R is:

$$R = \begin{vmatrix} 1 - 2(y^2 + z^2) & 2(xy - zw) & 2(xz + yw) \\ 2(xy + zw) & 1 - 2(x^2 + z^2) & 2(yz - xw) \\ 2(xz - yw) & 2(yz + xw) & 1 - 2(x^2 + y^2) \end{vmatrix}$$

Continuing: the 90° Z-rotation quaternion $(0.707, 0, 0, 0.707)$

$w=0.707, x=0, y=0, z=0.707$

$R[0][0] = 1 - 2(0^2 + 0.707^2) = 1 - 2(0.5) = 0$

$R[0][1] = 2(0 \cdot 0 - 0.707 \cdot 0.707) = 2(-0.5) = -1$

$R[1][0] = 2(0 \cdot 0 + 0.707 \cdot 0.707) = 2(0.5) = 1$

$R[1][1] = 1 - 2(0^2 + 0.707^2) = 0$

... (z row/col stays identity)

$R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ A point at (1, 0, 0) — 1m ahead — becomes $R @ (1,0,0) = (0, 1, 0)$ — 1m to the left.
Correct for a 90° left yaw.

Why quaternions instead of just using Euler angles (roll, pitch, yaw)? Euler angles suffer from "gimbal lock" — a configuration where two axes align and you lose a degree of freedom. Quaternions don't have this problem and are also faster to compose (multiply two quaternions instead of multiplying three matrices).

CHAPTER 4

Forward and Inverse Transforms

Once we have the rotation matrix R and translation vector t , we can move points between frames. There are two directions:

Forward transform — sensor frame $\rightarrow$ world frame

If a sensor is at position t with orientation R relative to some parent frame, then a point p expressed in the sensor frame becomes, in the parent frame:

$$p_{\text{parent}} = R @ p_{\text{sensor}} + t$$

Intuitively: R rotates the point to align with the parent's axes, then t shifts it to where the sensor actually sits in the parent frame.

Inverse transform — world frame $\rightarrow$ sensor frame

Going the other direction, given a point in the parent frame, express it in the sensor frame:

$$p_{\text{sensor}} = R^T @ (p_{\text{parent}} - t)$$

We subtract the translation first (un-shift), then apply the transposed rotation matrix. For rotation matrices, the transpose equals the inverse ($R^T = R^{-1}$) because rotation matrices are orthogonal — rotating then un-rotating is identity.

Example: A LiDAR sensor mounted 1.8m above the car centre, no rotation.

$R = \text{identity}$, $t = [0, 0, 1.8]$

A LiDAR point at $[10, 5, 0]$ (in LiDAR frame) in ego frame:

$$p_{\text{ego}} = I @ [10, 5, 0] + [0, 0, 1.8] = [10, 5, 1.8]$$

Makes sense — the point is 10m ahead, 5m left, and since the LiDAR is 1.8m up, the same physical point is 1.8m above the ground in the ego frame.

These two functions — `transform_points()` and `inverse_transform_points()` — are the only mathematical primitives the entire pipeline is built on.

LiDAR → Camera Projection — The Full Chain

This is the centrepiece of the pipeline. We want to draw each LiDAR point on the corresponding camera image, coloured by how far away it is. To do this, we need to figure out: *which pixel does this 3D point land on?*

The answer requires moving the point through four coordinate frames in sequence:

```

LiDAR sensor frame
    ↓ forward transform (LiDAR calibration)
Ego vehicle frame
    ↓ forward transform (ego pose)
World frame
    ↓ inverse transform (camera ego pose)
Camera ego frame (same car, but at camera's timestamp)
    ↓ inverse transform (camera calibration)
Camera sensor frame
    ↓ pinhole projection (intrinsic matrix)
Image pixel (u, v)
  
```

Why go all the way through the world frame? Because the LiDAR and cameras don't fire at exactly the same instant. The LiDAR might have taken its sweep a few milliseconds before the cameras. In those milliseconds the car moved. By converting both to the world frame (using each sensor's own ego pose), we account for that motion before comparing them.

Step-by-step in code

In `lidar_points_to_camera()` :

- LiDAR frame → ego frame:** `transform_points(points, lidar.calibrated_sensor["rotation"], lidar.calibrated_sensor["translation"])`
- Ego frame → world frame:** `transform_points(points, lidar.ego_pose["rotation"], lidar.ego_pose["translation"])`
- World frame → camera's ego frame:** `inverse_transform_points(points, camera.ego_pose["rotation"], camera.ego_pose["translation"])`
- Camera's ego frame → camera sensor frame:** `inverse_transform_points(points, camera.calibrated_sensor["rotation"], camera.calibrated_sensor["translation"])`

After step 4 we have each point in the camera's own 3D frame, with Z pointing forward out of the lens. Points with $Z \leq 0$ are behind the camera and are discarded.

Numerical walkthrough — single point, simplified numbers

Say a LiDAR point is at $\mathbf{p_lidar} = [10, 0, 0]$ (10m straight ahead of the LiDAR).

Step 1 — LiDAR to ego

LiDAR calibration: translation = $[0, 0, 1.8]$, rotation = identity

$$\mathbf{p_ego} = [10, 0, 0] + [0, 0, 1.8] = [10, 0, 1.8]$$

Step 2 — Ego to world

Car is at world position $[500, 200, 0]$, heading north (identity rotation for simplicity)

$$\mathbf{p_world} = [10, 0, 1.8] + [500, 200, 0] = [510, 200, 1.8]$$

Step 3 — World to camera ego (camera captured slightly later, car moved 0.1m forward)

Camera ego translation = $[500.1, 200, 0]$

$$\mathbf{p_cam_ego} = [510, 200, 1.8] - [500.1, 200, 0] = [9.9, 0, 1.8]$$

Step 4 — Camera ego to camera sensor

Front camera is at $[1.5, 0, 1.5]$ on the car, pointing forward (identity rotation)

$$\mathbf{p_cam} = [9.9, 0, 1.8] - [1.5, 0, 1.5] = [8.4, 0, 0.3]$$

The point is now in the camera frame: 8.4m ahead, 0m left, 0.3m above the optical axis.

Depth = 8.4m (the Z component in camera frame).

The Pinhole Camera Model — 3D to 2D

Once we have a point in the camera's own 3D frame (X, Y, Z), we need to figure out which pixel it projects to. This is the **pinhole camera model**.

Imagine a pinhole camera: light from a 3D point passes through a tiny hole and hits the image plane. Geometry tells us the pixel location:

$$\begin{aligned}u &= f_x \cdot (X/Z) + c_x \\v &= f_y \cdot (Y/Z) + c_y\end{aligned}$$

Where:

- **f_x, f_y** — focal length in pixels. Describes how strongly the lens "zooms". A larger focal length means a narrower field of view and objects appear bigger.
- **c_x, c_y** — the principal point. Where the optical axis hits the image plane — usually close to the image centre.
- **$X/Z, Y/Z$** — the angular position of the point. Dividing by Z is the perspective divide — farther objects are divided by a larger Z, making them appear smaller.

In matrix form, these four numbers form the **intrinsic matrix K**:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Projection: $[u \cdot Z, v \cdot Z, Z] = K @ [X, Y, Z]$
Then divide by Z to get pixel (u, v).

Continuing the example — projecting [8.4, 0, 0.3] to a pixel

nuScenes front camera typical intrinsics: $f_x = f_y = 1260$, $c_x = 807$, $c_y = 495$

Point in camera frame: $X=8.4, Y=0, Z=0.3$... wait, $Z=0.3$ means only 30cm in front of the lens — that's inside the hood. Let's use a more realistic depth.

Say the final camera-frame point is **[0.5, -0.2, 10.0]** (half a metre right, 20cm below optical axis, 10m ahead).

$$u = 1260 \cdot (0.5 / 10) + 807 = 63 + 807 = \mathbf{870 \text{ px}}$$

$$v = 1260 \cdot (-0.2 / 10) + 495 = -25.2 + 495 = \mathbf{470 \text{ px}}$$

Image is 1600×900. $u=870$, $v=470$ — well within bounds, slightly right of centre, slightly below centre. That's where the point gets drawn.

In `project_camera_points()`, this is done for all points at once using matrix operations, then we filter out any pixel coordinates outside the image bounds.

Why points behind the camera must be discarded

If $Z \leq 0$, the point is behind the camera. The formula still produces a pixel coordinate, but it's geometrically meaningless — a point 1m behind the lens doesn't appear on the image at all. The code discards all points with $Z \leq 1.0\text{m}$ (a small buffer for numerical safety).

Drawing the points

In the old approach, circles were drawn on the full-resolution image (1600×900) and then the image was downsampled to the tile width (~360px). A 1px circle on a 1600px image becomes a fraction of a pixel after downscaling — nearly invisible.

The fix: downscale the image first, then scale the UV coordinates by the same factor, then draw circles on the small tile. A radius=2 dot on a 360px image is clearly visible.

```
scale = tile_width / original_width          # e.g. 360/1600 = 0.225
uv_scaled = (uv * scale).astype(int)        # scale every pixel coord
draw circles of radius 2 on the resized tile
```

Depth Colouring — Making Distance Visible

Each projected LiDAR point is coloured by its depth (the Z value in camera frame). The colour mapping uses OpenCV's

`COLORMAP_TURBO` : red = close, blue = far.

The normalisation in `depth_to_bgr()` :

```
normalized = 255 × (1 - depth / max_depth)

depth=0m → normalized=255 → bright red (close)
depth=40m → normalized=127 → green (mid-range)
depth=80m → normalized=0 → dark blue (far, at the colour map limit)
```

The `1 - ...` inversion means high depth gets a low value. TURBO's low values are blue, high values are red, so the effect is: nearby points are hot/red, distant points are cool/blue. This matches the intuitive "danger zone" colour convention used in automotive displays.

CHAPTER 8

Bird's Eye View — Looking Down from Above

The BEV gives an overhead map of the LiDAR scan. It's simpler than the camera projection because we only need one transform, not four: we go from LiDAR sensor frame directly to the ego vehicle frame.

```
LiDAR sensor frame
    ↓ forward transform (LiDAR calibration only)
Ego vehicle frame ← BEV is rendered here
```

In the ego frame, X points forward (the car's driving direction), Y points left, and Z points up. The BEV is a top-down view, so we look straight down the Z-axis and map (X, Y) to image (row, col).

Metric-to-pixel mapping

The BEV covers a square patch of ground around the car. In our code: $x_range = (-50, 50)$, $y_range = (-50, 50)$, resolution = 0.25 m/cell. That gives a 400×400 grid (100m / 0.25 = 400 cells per side).

```
Image size:
    height = (x_max - x_min) / resolution = (50 - (-50)) / 0.25 = 400 px
    width  = (y_max - y_min) / resolution = (50 - (-50)) / 0.25 = 400 px

For a point at ego (X, Y):
    row = (x_max - X) / resolution
    col = (y_max - Y) / resolution
```

The subtractions flip the axes so that:

- X=50 (50m ahead) → row=0 (top of image) — *forward is up*
- X=0 (the car) → row=200 (middle) — *ego is at centre*
- Y=50 (50m left) → col=0 (left of image) — *left stays left*

Where does a specific point land?

A parked car is 30m ahead of us and 5m to the right in ego frame: X=30, Y=-5

```
row = (50 - 30) / 0.25 = 20 / 0.25 = 80 px from top
col = (50 - (-5)) / 0.25 = 55 / 0.25 = 220 px from left
```

Our car (X=0, Y=0):

row = (50 - 0) / 0.25 = **200**, col = (50 - 0) / 0.25 = **200** — exactly centre.

Z-height colouring in BEV

In the BEV we colour points by their Z height (how far above the ground), not by depth. The Z range used is [-3.0, 5.0] metres:

```
z_norm = (Z - z_min) / (z_max - z_min) = (Z + 3) / 8

Z = -3m (below ground, e.g. scan noise) → z_norm = 0 → dark blue
Z = 0m (ground level) → z_norm = 0.375 → green
Z = 1.8m (car roof height) → z_norm = 0.6 → orange/yellow
Z = 5m (tall buildings, trees) → z_norm = 1 → bright red
```

This is useful because you can visually distinguish ground returns (green), vehicles (yellow), and tall structures (red) just from the colour of each point in the BEV.

The ego vehicle marker

The white rectangle with an arrow at the centre of the BEV represents our car. The size is hardcoded to approximately 2m wide × 4.5m long (a typical car footprint), and the arrow points in the +X direction (forward). Since the ego frame by definition has the car at the origin heading forward, the marker is always at the centre pointing up.

The metric grid

The faint grid lines are drawn every 10 metres. The code iterates over multiples of 10 within the X and Y range, converts each to a pixel row/col, and draws a line there. Labels like "10m", "20m" are added alongside. The origin lines (x=0 and y=0) are drawn slightly brighter to give a clear reference.

Putting It All Together — What Happens Per Frame

For each keyframe in the scene, here is the complete sequence of operations:

1. **Load the LiDAR point cloud** from .pcd.bin $\rightarrow$ $N \times 3$ array of (x, y, z) floats in LiDAR sensor frame.
2. **Load the six camera images** from disk (JPEG, typically 1600×900).
3. **Load calibration and pose data** from the JSON tables for this specific timestamp.
4. **For each of the six cameras:**
 - Run the four-step transform chain: LiDAR $\rightarrow$ ego $\rightarrow$ world $\rightarrow$ camera ego $\rightarrow$ camera sensor.
 - Discard all points with $Z \leq 1\text{m}$ (behind camera or too close).
 - Project the rest to (u, v) pixels using the intrinsic matrix.
 - Discard pixels outside image bounds.
 - Downscale the camera image to tile width.
 - Scale the UV coordinates by the same factor.
 - Draw depth-coloured circles at each projected point.
5. **Tile the six camera images** into a 2×3 grid (top row: FL, F, FR; bottom row: BL, B, BR).
6. **For the BEV:** run the single LiDAR $\rightarrow$ ego transform, map to pixels, colour by Z, draw the grid and ego marker.

The result is a complete situational picture: you can see in all six directions simultaneously (cameras view), understand exactly where every LiDAR return sits in 3D (lidar projection view), and have a clean overhead map of the environment (BEV).

Quick Reference — Key Numbers

Quantity	Value	Where it appears
Keyframe rate	2 Hz (one per 0.5s)	Dataset structure
Sweep rate (cameras)	~12 Hz	iter_camera_sweep_frames
Samples per scene	~40	sample["next"] linked list length
Camera resolution	1600 × 900 px	nuScenes standard
Focal length (typical)	$f_x = f_y \approx 1260$ px	calibrated_sensor intrinsics
LiDAR points per sweep	~34,000	load_lidar_points output size
BEV coverage	100m × 100m	x_range, y_range = (-50, 50)
BEV resolution	0.25 m/cell → 400×400 px	render_lidar_bev
BEV Z range	-3m to +5m	z_range filter
Max depth for colouring	80m	depth_to_bgr max_depth